2026-7-4 Linux开发工具--代码编辑器

- 1. Linux 环境下的代码编辑器 vim
  - 1.1 Linux 开发工具为什么是分开的
    - 1.1.1 IDE 与独立工具
    - 1.1.2 vim 在开发流程中的位置
  - 1.2 vi 与 vim 的关系
    - 1.2.1 vim 是 vi 的增强版
    - 1.2.2 vim 的两个核心特点
    - 1.2.3 为什么 vim 要使用命令
- 2. vim 的模式与基本操作
  - 2.1 vim 的三个基本模式
    - 2.1.1 命令模式
    - 2.1.2 插入模式
    - 2.1.3 底行模式
  - 2.2 vim 的扩展模式
    - 2.2.1 可视块模式
    - 2.2.2 替换模式
  - 2.3 模式转换
    - 2.3.1 常用转换路径
    - 2.3.2 最重要的是先按 Esc
  - 2.4 打开文件与退出
    - 2.4.1 打开文件
    - 2.4.2 第一次完整编辑
    - 2.4.3 保存与退出
- 3. 命令模式
  - 3.1 光标移动
    - 3.1.1 基本方向键 hjkl
    - 3.1.2 跳到文件开头和结尾
    - 3.1.3 跳到行首和行尾
    - 3.1.4 按单词移动
    - 3.1.5 按列和屏幕移动
    - 3.1.6 重新打开时恢复位置
  - 3.2 删除、修改与撤销
    - 3.2.1 删除字符
    - 3.2.2 删除行
    - 3.2.3 单字符替换

        - 3.2.4 连续替换模式
        - 3.2.5 大小写转换
        - 3.2.6 修改单词
        - 3.2.7 撤销和重做
    - 3.3 复制、剪切与粘贴
        - 3.3.1 复制
        - 3.3.2 粘贴
        - 3.3.3 与数量的组合
    - 3.4 批量注释与取消注释
        - 3.4.1 批量添加注释
        - 3.4.2 批量取消注释
        - 3.4.3 批量为多行增加相同内容
- 4. 底行模式
    - 4.1 进入底行模式
    - 4.2 显示行号与跳行
        - 4.2.1 显示行号
        - 4.2.2 跳到指定行
    - 4.3 搜索
        - 4.3.1 向后搜索
        - 4.3.2 向前搜索
        - 4.3.3 下一个和上一个匹配
        - 4.3.4 搜索后的操作
    - 4.4 批量替换
        - 4.4.1 基本格式
        - 4.4.2 删除字符串
        - 4.4.3 逐项确认
        - 4.4.4 更换分隔符
        - 4.4.5 只替换当前行
    - 4.5 执行 Shell 命令
        - 4.5.1 临时执行
        - 4.5.2 执行后的注意点
    - 4.6 分屏与多文件编辑
        - 4.6.1 垂直分屏
        - 4.6.2 切换窗口
        - 4.6.3 分屏的用途
- 5. 保存、退出与权限问题
    - 5.1 基本保存命令

        - 5.1.1 保存
        - 5.1.2 保存并退出
        - 5.1.3 只退出
    - 5.2 强制命令不等于拥有权限
        - 5.2.1 强制退出
        - 5.2.2 强制保存
        - 5.2.3 保存并强制退出
    - 5.3 没有权限时的现象
        - 5.3.1 没有读权限
        - 5.3.2 没有写权限
    - 5.4 快速保存退出
- 6. vim 异常退出与 swap 文件
    - 6.1 为什么会产生临时文件
        - 6.1.1 swap 文件
        - 6.1.2 如何观察
    - 6.2 非正常退出的提示
    - 6.3 如何处理临时文件
        - 6.3.1 先判断是否需要恢复
        - 6.3.2 确认不需要时再删除
        - 6.3.3 为什么不要直接忽略提示
    - 6.4 避免多个 vim 同时编辑同一文件
- 7. vim 配置
    - 7.1 配置文件的位置
        - 7.1.1 用户配置
        - 7.1.2 系统配置
        - 7.1.3 配置的本质
    - 7.2 常用基础配置
        - 7.2.1 一个简洁的 .vimrc
        - 7.2.2 各选项含义
        - 7.2.3 只临时设置
        - 7.2.4 查看配置是否生效
    - 7.3 使用插件
        - 7.3.1 插件目录
        - 7.3.2 安装插件的一般步骤
        - 7.3.3 文件浏览器示例
        - 7.3.4 插件配置仍写入 .vimrc
    - 7.4 常见配置误区

        - 7.4.1 误以为 w! 可以绕过权限
        - 7.4.2 误把插入模式当默认模式
        - 7.4.3 误以为退出后仍能撤销
        - 7.4.4 直接删除 swap 文件
        - 7.4.5 配置文件写错位置
- 8. vim 命令速查与练习
    - 8.1 模式速查
    - 8.2 光标移动速查
    - 8.3 编辑速查
    - 8.4 底行模式速查
    - 8.5 建议练习
        - 8.5.1 基础操作
        - 8.5.2 搜索与替换
        - 8.5.3 批量编辑
        - 8.5.4 配置与恢复
    - 8.6 vim 学习主线

1. Linux 环境下的代码编辑器 vim

1.1 Linux 开发工具为什么是分开的

1.1.1 IDE 与独立工具

常见的 IDE（集成开发环境）把编辑、编译、调试等功能放在同一个界面中。Linux 课堂更强调将工具拆开理解：

```
写代码 → 编辑器
编译代码 → 编译器
调试程序 → 调试器
批量构建 → 构建工具
管理版本 → 版本控制工具
```

这样做的好处是每个工具职责明确，可以单独使用，也可以组合进项目流程。

1.1.2 vim 在开发流程中的位置

vim 是 Linux 环境中常用的终端代码编辑器，主要解决：

```
打开文件
→ 输入和修改代码
→ 搜索、替换与批量编辑
→ 保存
→ 退出
```

本篇只整理 vim，不展开编译、调试、构造工具或版本控制。

1.2 vi 与 vim 的关系

1.2.1 vim 是 vi 的增强版

vi 和 vim 都是多模式编辑器。vim 兼容 vi 的常用操作，并增加了：

- 语法高亮。
- 更丰富的可视化和多窗口操作。
- 更多编辑命令。
- 可配置的编辑环境。
- 跨平台支持。

Linux 中执行：

```
vim --version
```

可以查看当前 vim 的版本和支持的特性。

1.2.2 vim 的两个核心特点

第一，启动快：

```
vim file.c
```

可以直接在终端中打开文件，不需要等待大型图形界面。

第二，采用多模式编辑。不同模式承担不同任务，模式之间通过固定按键切换。

1.2.3 为什么 vim 要使用命令

普通编辑器通常把字母键直接用于输入文字。vim 把不同操作放进不同模式：

- 在命令模式下，按键用于移动、删除、复制和跳转。
- 在插入模式下，按键才作为文字输入。
- 在底行模式下，按键用于保存、搜索、替换和执行命令。

快捷键和编辑命令本质上都是对操作的自动化与批量化。熟练以后，可以用少量按键完成大量编辑工作。

vim 的学习重点不是死记所有按键，而是理解：

```
当前是什么模式
→ 在这个模式里能做什么
→ 用哪个键转到下一个模式
```

2. vim 的模式与基本操作

2.1 vim 的三个基本模式

2.1.1 命令模式

打开 vim 后，默认进入正常模式、普通模式或命令模式：

```
vim text.c
```

命令模式主要用于：

- 移动光标。
- 删除字符或整行。
- 复制和粘贴。
- 替换字符。
- 撤销与重做。
- 进入插入模式。
- 进入底行模式。
- 进入可视块模式。

进入 vim 后直接按字母，通常不会输入文字，而是执行对应命令。

2.1.2 插入模式

只有插入模式才能输入普通文字。

从命令模式进入插入模式：

按键	作用
`i`	在光标前开始输入
`a`	在光标后开始输入
`o`	在当前行下方新建一行并输入

从插入模式返回命令模式：

```
Esc
```

插入模式是日常编辑中最常使用的模式，但保存、搜索和批量操作通常在命令模式或底行模式完成。

2.1.3 底行模式

在命令模式按：

```
:
```

进入底行模式。

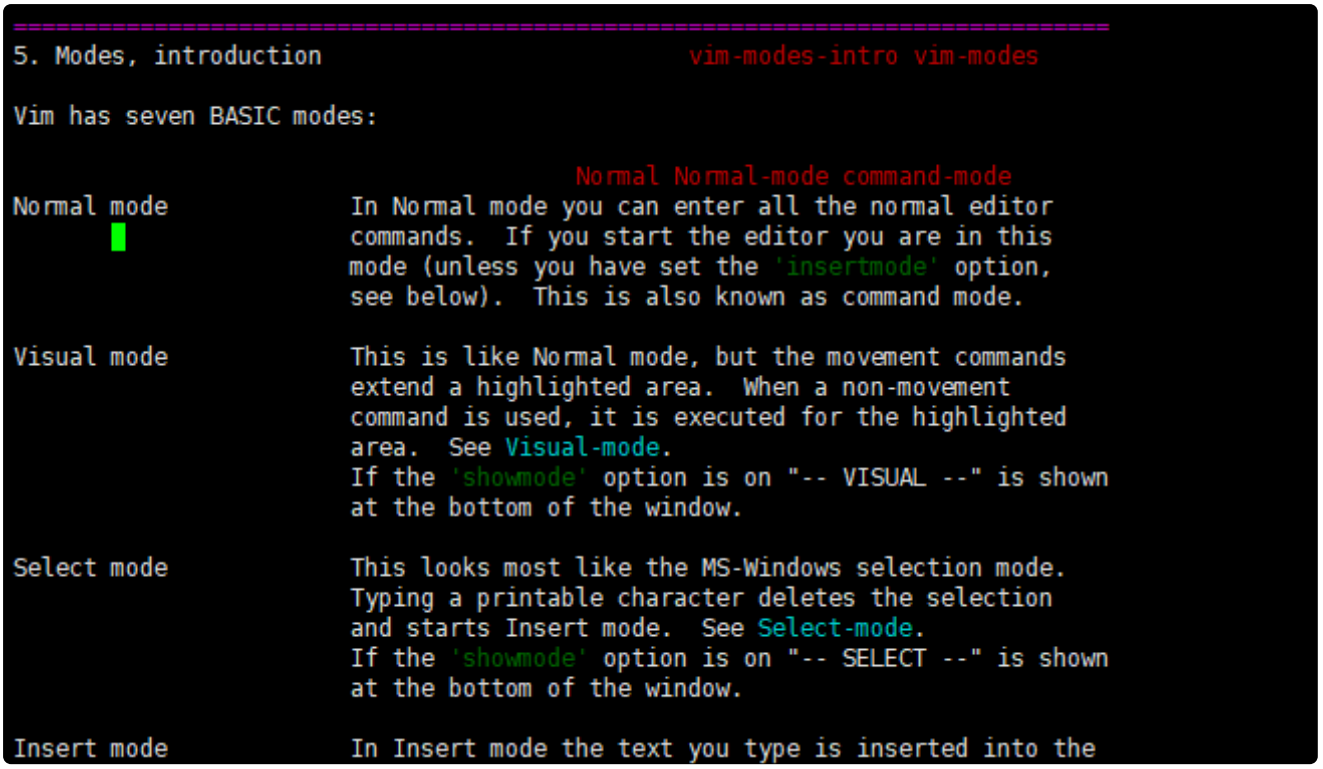
底行模式常用于：

- 保存和退出。
- 搜索。
- 批量替换。
- 显示行号。

- 分屏打开文件。
- 临时执行 Shell 命令。

如果不知道当前 vim 支持哪些模式，可以在 vim 底行模式执行：

```
:help vim-modes
```



2.2 vim 的扩展模式

2.2.1 可视块模式

在命令模式按：

```
Ctrl+v
```

进入 VISUAL BLOCK 模式，用来选择矩形文本区域。它常用于同时编辑多行的相同列，例如批量添加或删除注释符。

按 `Esc` 返回命令模式。

2.2.2 替换模式

在命令模式按：

```
R
```

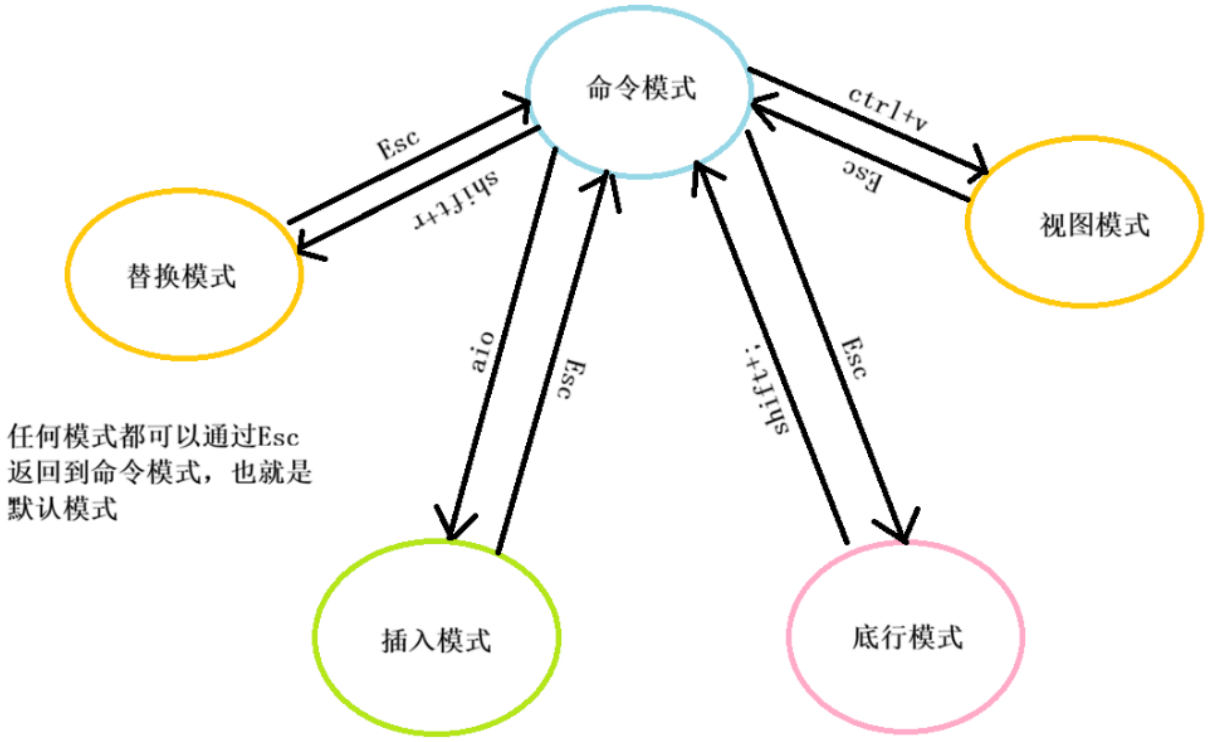
进入替换模式。继续输入字符会依次覆盖光标后的文字，按 `Esc` 返回命令模式。

替换模式适合连续覆盖一段固定长度的内容，与单字符替换命令 `r` 不同。

2.3 模式转换

2.3.1 常用转换路径

```
命令模式 --i/a/o--> 插入模式
插入模式 --Esc--> 命令模式
命令模式 --:--> 底行模式
底行模式 --Esc或执行完成--> 命令模式
命令模式 --Ctrl+v--> 可视块模式
命令模式 --R--> 替换模式
```



2.3.2 最重要的是先按 Esc

当不确定自己处于什么模式时，先按：

```
Esc
```

通常可以回到命令模式。然后再执行保存、移动或其他命令。

插入模式和底行模式不能随意直接互相切换。一般先回到命令模式，再进入另一个模式。

2.4 打开文件与退出

2.4.1 打开文件

```
vim text.c
```

如果文件不存在，vim 可以在保存时创建它。

指定打开后的光标位置：

```
vim text.c +9
```

这会让 vim 打开文件后把光标定位到第 9 行。

2.4.2 第一次完整编辑

可以先用一个最小练习熟悉模式转换：

1. 打开文件：

```
vim hello.c
```

2. 进入插入模式：

```
i
```

3. 输入一小段文本，例如：

```c
#include <stdio.h>

int main(void)
{
    printf("hello vim\n");
    return 0;
}
```

4. 按 `Esc` 回到命令模式。
5. 输入 `:wq` 保存并退出。
6. 再次执行 `vim hello.c`，检查文件是否已经保存。

这个练习只关注 vim 的打开、输入、保存和退出，不涉及如何编译程序。

2.4.3 保存与退出

在底行模式：

命令	作用
`:w`	保存
`:q`	退出
`:wq`	保存并退出
`:x`	需要时保存并退出
`:q!`	不保存并强制退出

也可以在命令模式下输入：

```
ZZ
```

这会保存并退出 vim。

3. 命令模式

3.1 光标移动

3.1.1 基本方向键 hjkl

vim 使用：

按键	方向
`h`	左
`j`	下
`k`	上
`l`	右

为什么有了方向键还要用 hjkl?

1. 右手通常停留在主键区，使用 hjkl 移动比反复移动到方向键更快。
2. 早期终端键盘没有现代方向键，vim 保留了旧使用习惯。

现代环境中方向键当然可以使用，但 hjkl 更容易与 vim 的其他命令组合。

3.1.2 跳到文件开头和结尾

```
gg    跳到文件开头
G     跳到文件末尾
nG    跳到第 n 行
```

例如：

```
15G
```

跳到第 15 行。

3.1.3 跳到行首和行尾

命令	作用
`0`	跳到当前行最前面
`^`	跳到当前行第一个非空白字符

命令	作用
`$`	跳到当前行行尾

课堂中把 `0`、`^`、`$` 理解为行内锚点命令。

3.1.4 按单词移动

命令	作用
`w`	跳到下一个单词开头
`nw`	连续跳 n 个单词
`b`	回到上一个单词开头
`nb`	连续向左跳 n 个单词
`e`	跳到当前或下一个单词末尾

3.1.5 按列和屏幕移动

```
nl
```

表示在原位置基础上向右移动 n 个字符位置，例如 `5l`。它是数量修饰命令，不是直接跳到绝对第 n 列。

翻屏：

命令	作用
`Ctrl+b`	向文件开头方向翻一页
`Ctrl+f`	向文件末尾方向翻一页
`Ctrl+u`	向文件开头方向翻半页
`Ctrl+d`	向文件末尾方向翻半页

3.1.6 重新打开时恢复位置

vim 通常会记住上次退出时光标所在的位置。再次打开同一文件时，可以把光标恢复到附近位置。

也可以直接指定初始行号：

```
vim file.c +20
```

3.2 删除、修改与撤销

3.2.1 删除字符

命令	作用
`x`	删除光标所在字符
`nx`	从光标位置向右删除 n 个字符
`X`	删除光标前一个字符
`nX`	向左删除 n 个字符

3.2.2 删除行

命令	作用
`dd`	删除当前行
`ndd`	从当前行开始删除 n 行

vim 的删除操作通常也会把内容放入内部缓冲区，因此可以再通过粘贴命令使用。

3.2.3 单字符替换

```
r
```

按 `r` 后再输入一个字符，会替换光标所在位置的单个字符。

例如，把光标停在 `+` 上，按 `r`，再输入 `-`，即可快速把 `+` 替换为 `-`。

3.2.4 连续替换模式

```
R
```

进入替换模式，之后输入的字符依次覆盖原文，直到按 `Esc`。

3.2.5 大小写转换

对单个字符使用：

```
~
```

可以切换大小写。可视模式中也可以先选中文本，再执行大小写转换。

3.2.6 修改单词

命令	作用
`cw`	修改一个单词
`c3w`	修改 3 个单词

3.2.7 撤销和重做

命令	作用
`u`	撤销上一步操作
`Ctrl+r`	重做被撤销的操作

只要 vim 还没有退出，通常可以在编辑会话中反复撤销和重做。退出以后，普通撤销历史通常不会保留。

3.3 复制、剪切与粘贴

3.3.1 复制

命令	作用
`yy`	复制当前行
`nyy`	复制从当前行开始的 n 行
`yw`	复制从光标到当前单词末尾
`nyw`	复制 n 个单词

3.3.2 粘贴

命令	作用
`p`	粘贴到光标后或当前行下方
`P`	粘贴到光标前或当前行上方

`p` 与复制、删除和剪切命令配合非常频繁。

3.3.3 与数量的组合

vim 命令常常可以写成：

```
数字 + 操作
```

例如：

```
5yy     复制 5 行
3dd     删除 3 行
10j     向下移动 10 行
8x      向右删除 8 个字符
```

理解“数量修饰命令”比单独记忆每个组合更重要。

3.4 批量注释与取消注释

3.4.1 批量添加注释

假设要给多行前面添加 `//`：

1. 在命令模式按 `Ctrl+v` 进入可视块模式。
2. 用 hjkl 选择多行的同一列。
3. 按 `I` 进入块插入模式。
4. 输入 `//`。
5. 按 `Esc`，选中的多行会同时插入注释符。

3.4.2 批量取消注释

1. 按 `Ctrl+v` 进入可视块模式。
2. 选中要删除的注释符所在列。
3. 按 `d` 删除。
4. 按 `Esc` 返回命令模式。

3.4.3 批量为多行增加相同内容

可视块加插入不只可以添加注释，也可以为多行统一增加缩进、前缀或变量名。

4. 底行模式

4.1 进入底行模式

先确保处于命令模式，然后按：

```
:
```

进入底行模式。

底行命令执行后通常自动回到命令模式；部分搜索命令则保留搜索状态。

4.2 显示行号与跳行

4.2.1 显示行号

```
:set nu
```

关闭行号：

```
:set nonu
```

4.2.2 跳到指定行

```
:15
```

跳到第 15 行。

4.3 搜索

4.3.1 向后搜索

```
/keyword
```

输入关键字后按回车，vim 从当前位置向后搜索。

4.3.2 向前搜索

```
?keyword
```

vim 从当前位置向前搜索。

4.3.3 下一个和上一个匹配

命令	作用
`n`	沿当前搜索方向查找下一个
`N`	沿反方向查找上一个

4.3.4 搜索后的操作

搜索到目标后，可以继续使用：

- `n` 或 `N` 继续跳转。
- `r` 替换单个字符。
- `cw` 修改当前单词。
- `x` 删除字符。

4.4 批量替换

4.4.1 基本格式

```
:%s/old/new/g
```

各部分的含义：

部分	含义
`%`	对整篇文件操作
`s`	substitute，替换
`old`	原字符串
`new`	新字符串
`g`	每一行所有匹配都替换

4.4.2 删除字符串

把替换内容留空：

```
:%s/unwanted//g
```

这会删除整篇文件中所有 `unwanted`。

4.4.3 逐项确认

```
:%s/old/new/gc
```

末尾的 `c` 表示每次替换前请求确认，适合不希望一次性改完的情况。

4.4.4 更换分隔符

如果字符串中包含 `/`，可以改用其他分隔符：

```
:%s#/old/path#/new/path#g
```

4.4.5 只替换当前行

省略前面的 `%`：

```
:s/old/new/g
```

表示只对当前行执行替换。

4.5 执行 Shell 命令

4.5.1 临时执行

```
:!pwd
:!ls -l
```

`!` 表示暂时调用 Shell 命令，执行完按回车返回 vim。

4.5.2 执行后的注意点

Shell 命令输出通常只用于查看，不能替代在普通终端中修改项目状态。执行会修改文件的命令前，应先明确当前目录和影响范围。

4.6 分屏与多文件编辑

4.6.1 垂直分屏

```
:vs header.h
```

`vs` 是 vertical split 的常用缩写，可以创建垂直分屏并打开文件。

4.6.2 切换窗口

```
Ctrl+ww
```

在不同分屏之间循环切换。

4.6.3 分屏的用途

分屏常用于同时查看：

- 源文件和头文件。
- 函数定义和调用位置。
- 两个需要对照修改的文件。

5. 保存、退出与权限问题

5.1 基本保存命令

5.1.1 保存

```
:w
```

只保存，不退出。

5.1.2 保存并退出

```
:wq
```

也可以使用：

```
ZZ
```

5.1.3 只退出

```
:q
```

如果文件修改尚未保存，vim 会拒绝退出：

```
No write since last change
```

此时应先保存，或者明确选择不保存并退出。

5.2 强制命令不等于拥有权限

5.2.1 强制退出

```
:q!
```

表示放弃未保存的修改并退出。

5.2.2 强制保存

```
:w!
```

尝试强制写入。但这里的“强制”主要是覆盖 vim 的只读提示，不能绕过操作系统真正的文件系统权限。

如果当前用户对文件或父目录没有写权限，`w!` 仍然可能失败。

5.2.3 保存并强制退出

```
:wq!
```

尝试强制保存并退出。它也不能替代正确的文件权限。

5.3 没有权限时的现象

5.3.1 没有读权限

vim 可能打开一个空白界面，或者在读取文件时提示权限不足。

5.3.2 没有写权限

可以尝试进入编辑模式，但保存时失败。此时需要：

1. 确认当前用户是否有写权限。
2. 确认父目录是否允许完成相关操作。
3. 必要时使用正确的 `sudo` 方式编辑系统配置。

例如：

```
sudo vim /etc/system_file
```

不要通过放宽系统文件权限来“解决”保存失败。

5.4 快速保存退出

```
Esc
ZZ
```

是课堂中常用的保存并退出手势：

1. 按 `Esc` 回到命令模式。
2. 连续按两次大写 `Z`。

6. vim 异常退出与 swap 文件

6.1 为什么会产生临时文件

6.1.1 swap 文件

vim 编辑文件时，可能生成隐藏的临时文件，常见名称类似：

```
.text.c.swp
```

它表示 vim 正在编辑或曾经没有正常关闭该文件。

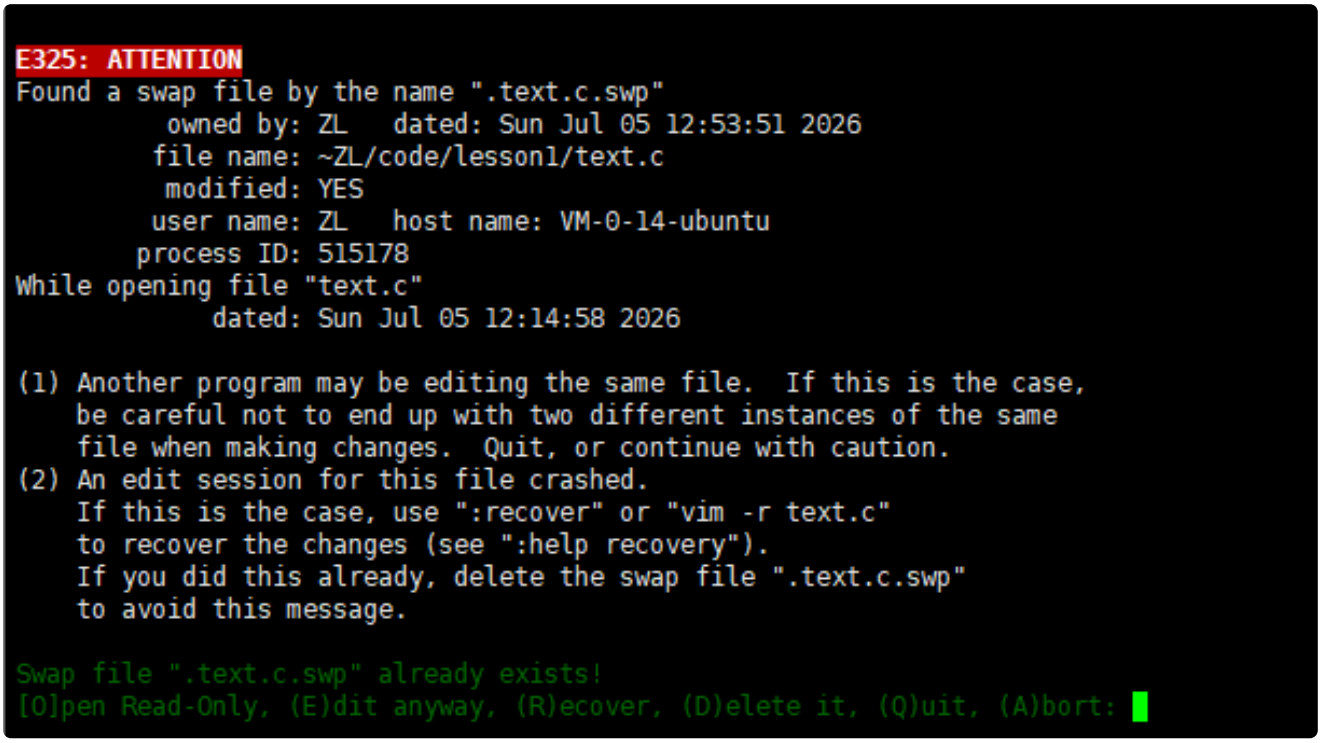
6.1.2 如何观察

```
ls -la
```

带 `-a` 可以看到以点开头的隐藏临时文件。

6.2 非正常退出的提示

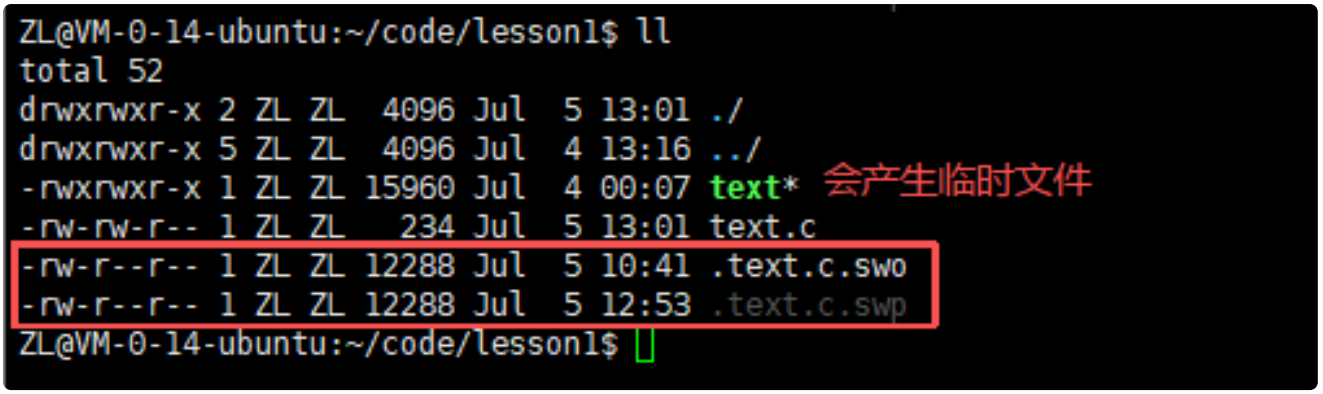
如果终端意外关闭、SSH 连接中断或 vim 被异常终止，再次打开同一文件时可能看到恢复界面：



界面通常会列出可用选项，例如：

- 恢复上次编辑内容。

- 直接打开原文件。
- 退出处理。
- 删除临时文件。



6.3 如何处理临时文件

6.3.1 先判断是否需要恢复

如果上次编辑的内容还有价值，应优先选择恢复，而不是直接删除 swap 文件。

也可以在启动 vim 时尝试恢复：

```
vim -r text.c
```

如果已经在 vim 中打开同一文件，可以使用底行命令查看或执行恢复：

```
:recover
```

恢复后：

1. 检查恢复出来的内容。
2. 立即保存原文件。
3. 确认保存成功后退出。
4. 再处理对应的 swap 文件。

6.3.2 确认不需要时再删除

如果确认原文件已经正常保存，没有未合并的修改，可以删除对应临时文件：

```
ls -la
rm .text.c.swp
```

只能删除自己创建的、确认无用的 swap 文件。不要删除其他人的临时文件或系统文件。

6.3.3 为什么不要直接忽略提示

swap 文件可能保存着尚未写入原文件的内容。直接删除可能丢失一次终端断开前的修改。

6.4 避免多个 vim 同时编辑同一文件

同一个文件被多个 vim 进程同时编辑时，容易产生冲突和临时文件。

如果确实需要多窗口编辑，应使用 vim 自带的分屏：

```
:vs file
```

或者在同一个 vim 实例中打开多个缓冲区，避免多个独立进程同时写同一文件。

7. vim 配置

7.1 配置文件的位置

7.1.1 用户配置

普通用户最常修改自己的：

```
~/.vimrc
```

编辑：

```
vim ~/.vimrc
```

`~` 表示当前用户的家目录。该配置只对当前用户生效。

7.1.2 系统配置

系统级 vim 配置通常位于：

```
/etc/vim/vimrc
/etc/vimrc
```

具体路径取决于 Linux 发行版。可以查看：

```
vim --version | head
```

系统级配置会影响使用该 vim 配置的多个账号。修改它通常需要 root 权限。

7.1.3 配置的本质

配置的本质是把选项写入 vim 启动时会读取的文件：

```
打开 vim
→ 读取系统配置
→ 读取当前用户的 ~/.vimrc
→ 应用选项
```

用户的个人配置通常不会自动影响其他用户。

7.2 常用基础配置

7.2.1 一个简洁的 .vimrc

```
syntax on
set number
set shiftwidth=4
set tabstop=4
set expandtab
set mouse=a
set ruler
set showmatch
set cursorline
```

7.2.2 各选项含义

配置	作用
`syntax on`	开启语法高亮
`set number`	显示绝对行号
`set shiftwidth=4`	自动缩进宽度为 4
`set tabstop=4`	Tab 显示宽度为 4
`set expandtab`	按 Tab 时插入空格
`set mouse=a`	启用鼠标支持
`set ruler`	显示光标行列位置
`set showmatch`	高亮匹配括号
`set cursorline`	高亮当前行

7.2.3 只临时设置

在底行模式直接输入：

```
:set number
:set nonumber
```

只影响当前 vim 会话。写入 `.vimrc` 后，后续启动才会持续生效。

7.2.4 查看配置是否生效

在 vim 中执行：

```
:set number?
:set tabstop?
```

末尾 `?` 表示查询该选项当前值。

7.3 使用插件

7.3.1 插件目录

传统 vim 插件常放在用户目录：

```
~/.vim/plugin
~/.vim/doc
~/.vim/autoload
```

建议使用主目录下的隐藏目录保存，不要把第三方插件内容随意写进系统公共目录。

7.3.2 安装插件的一般步骤

```
取得插件
→ 解压或放入插件目录
→ 在 ~/.vimrc 中写入配置
→ 重新启动 vim
→ 检查功能是否生效
```

安装前应阅读插件说明，确认它支持的 vim 版本、依赖和快捷键。

7.3.3 文件浏览器示例

课堂中曾使用文件浏览器和窗口管理插件。其核心作用是把文件树、标签列表和编辑窗口组合在一起，提升多文件编辑效率。

使用这类插件时要注意：

- 插件快捷键不要与自己常用命令冲突。
- 插件配置只对当前账号生效。
- 修改系统级配置需要 root 权限。
- 第三方插件数量过多可能拖慢启动。

7.3.4 插件配置仍写入 .vimrc

插件程序放在 `~/.vim` 下，插件的启用和快捷键通常仍然通过 `~/.vimrc` 配置。

如果配置写在用户家目录下，只有当前用户生效；如果需要所有用户共享同一设置，才考虑修改系统级 vimrc。

7.4 常见配置误区

7.4.1 误以为 w! 可以绕过权限

`:w!` 只能忽略 vim 的部分只读状态。它不能绕过文件系统写权限。

7.4.2 误把插入模式当默认模式

vim 启动后默认是命令模式。输入文字前要按 `i`、`a` 或 `o`。

7.4.3 误以为退出后仍能撤销

退出 vim 后，普通撤销历史通常不再保留。异常退出时 swap 文件可能提供恢复，但它不等同于完整的撤销历史。

7.4.4 直接删除 swap 文件

如果 swap 文件中可能还有未保存修改，直接删除可能造成内容丢失。应先恢复、比较，再决定删除。

7.4.5 配置文件写错位置

用户配置应写在 `~/.vimrc`。系统级配置路径依发行版而异，不能一概写成 `/etc/.vimrc`。

8. vim 命令速查与练习

8.1 模式速查

当前模式	操作	目标模式
命令模式	`i` / `a` / `o`	插入模式
插入模式	`Esc`	命令模式
命令模式	`:`	底行模式
底行模式	`Esc`	命令模式
命令模式	`Ctrl+v`	可视块模式
可视块模式	`Esc`	命令模式
命令模式	`R`	替换模式

当前模式	操作	目标模式
替换模式	`Esc`	命令模式

8.2 光标移动速查

```
h j k l   左 下 上 右
gg        文件开头
G         文件末尾
nG        第 n 行
0         行首
^         第一个非空白字符
$         行尾
w         下一个单词开头
b         上一个单词开头
e         单词末尾
nl        第 n 列
Ctrl+b    向后翻页
Ctrl+f    向前翻页
Ctrl+u    向后翻半页
Ctrl+d    向前翻半页
```

8.3 编辑速查

```
x         删除光标处字符
nx        向右删除 n 个字符
X         向左删除一个字符
nX        向左删除 n 个字符
dd        删除当前行
ndd       删除 n 行
r         替换一个字符
R         连续替换
cw        修改单词
u         撤销
Ctrl+r    重做
yy        复制当前行
nyy       复制 n 行
yw        复制单词
p         粘贴
P         向前粘贴
```

8.4 底行模式速查

```
:w          保存
:q          退出
:wq         保存并退出
:q!         不保存强制退出
:w!         尝试强制保存
:set nu     显示行号
:set nonu   隐藏行号
/keyword    向后搜索
?keyword    向前搜索
:noh        取消搜索高亮
:%s/old/new/g     全文替换
:%s/old/new/gc    逐项确认替换
:vs file    垂直分屏
:!pwd       临时执行 Shell 命令
```

8.5 建议练习

8.5.1 基础操作

1. 打开一个 C 文件，练习三种进入插入模式的方式。
2. 使用 `Esc` 在插入模式和命令模式之间切换。
3. 使用 hjkl 和单词移动命令定位代码。
4. 使用 `dd`、`yy`、`p` 重排代码行。
5. 使用 `u` 和 `Ctrl+r` 撤销、重做。

8.5.2 搜索与替换

1. 使用 `/` 查找一个变量名。
2. 使用 `n` 和 `N` 前后切换匹配结果。
3. 使用 `:%s/old/new/g` 替换一个变量名。
4. 使用 `:%s/old/new/gc` 逐项确认替换。

8.5.3 批量编辑

1. 使用 `Ctrl+v` 选中多行行首。
2. 使用 `I` 批量添加注释。
3. 再次使用 `Ctrl+v` 和 `d` 取消注释。

8.5.4 配置与恢复

1. 在 `~/.vimrc` 中开启行号和语法高亮。
2. 查询 `tabstop` 和 `number` 当前值。
3. 制造一次异常退出，观察 swap 文件。
4. 按提示恢复修改，并确认原文件正确保存。

8.6 vim 学习主线

```
理解工具分工
→ 掌握默认命令模式
→ 掌握插入模式和底行模式
→ 用数量、锚点和批量命令提高效率
→ 理解保存、退出和异常恢复
→ 通过 .vimrc 和插件形成自己的编辑环境
```

本文只整理 vim 编辑器相关内容，不展开其他 Linux 开发工具。